

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к индивидуальному домашнему заданию
по дисциплине
«Введение в нереляционные системы управления базами данных»
Тема: БД для наблюдений за животными

Студент(ы) гр. 3341

_____ Мальцев К.Л.

_____ Трофимов В.О.

_____ Кудин А.А.

Преподаватель

_____ Корытов П.В.

Санкт-Петербург

2026

ЗАДАНИЕ

Студент(ы)

Мальцев К.Л.

Трофимов В.О.

Кудин А.

Группа 3341

Тема проекта:

БД для наблюдений за животными

Исходные данные:

Задача - создать веб-приложение, в котором можно делиться своими наблюдениями за животными (фото, видео, отметки на карте, лайки, комментарии).

Содержание пояснительной записки:

«Содержание»

«Введение»

«Качественные требования к решению»

«Сценарий использования»

«Модель данных»

«Разработка приложения»

«Вывод»

«Приложение»

Предполагаемый объем пояснительной записки:

Не менее 10 страниц.

Дата выдачи задания:

Дата сдачи реферата:

Дата защиты реферата:

Студент(ы) гр. 3341

_____ Мальцев К.Л.

_____ Трофимов В.О.

_____ Кудин А.А.

Преподаватель

_____ Кoryтов П.В.

АННОТАЦИЯ

В рамках курса разработано веб-приложение командой по теме «Приложение наблюдений» с использованием нереляционной СУБД. Приложение реализует интерфейсы и сценарии, описанные в макете и сценарии использования: страницы входа и регистрации, лента постов с фильтрами и возможностью массового импорта/экспорта, пошаговый процесс создания наблюдения (выбор типа, заголовок/описание, загрузка фото, место, теги), профиль пользователя с редактированием и статистикой, детальный просмотр поста с лайками и комментариями. Предусмотрены CRUD-операции для постов и комментариев, поиск и фильтрация ленты по типу, тегам, автору и диапазону дат, а также экспорт/импорт данных в формате JSON для администраторов.

<https://github.com/moevm/nsql1h26-anim>

ANNOTATION

As part of the course, a team developed a web application on the topic "Observation App" using a NoSQL database. The application implements the interfaces and use cases described in the UI mockup and scenarios: login and registration pages, a posts feed with filters and bulk import/export capabilities, a step-by-step observation creation flow (select type, title/description, upload photo, location, tags), a user profile with editing and statistics, and a detailed post view with likes and comments. CRUD operations for posts and comments are provided, along with search and filtering of the feed by type, tags, author, and date range, and JSON import/export functionality for administrators.

<https://github.com/moevm/nsql1h26-anim>

СОДЕРЖАНИЕ

Введение	7
1. Постановка задачи	9
1.1. Предлагаемое решение	9
1.2. Качественные требования к решению	9
2. Сценарии использования	11
2.1. Макет UI	18
3. Модель данных	19
3.1. Нереляционная модель данных (Neo4j)	19
3.2. Аналог модели данных для SQL СУБД	21
3.3. Сравнение моделей	24
4. Разработанное приложение	26
4.1. Краткое описание	26
4.2. Используемые технологии	26
4.3. Снимки экрана приложения	27
5. Выводы	28
Приложение А. Документация по сборке и развертыванию	29
Приложение Б. Инструкция для пользователя	30
Литература	31

Введение

Данные наблюдений естественно принимают графовую форму: наблюдения, пользователи, места и теги и их взаимосвязи (кто сделал наблюдение, где, с какими тегами, комментарии и лайки) моделируются как узлы (посты, пользователи, места, теги) и ребра (authored, located_at, tagged_with, commented_on, liked). Реляционные СУБД требуют множества таблиц и сложных JOIN-операций для представления и обхода таких связей, что снижает производительность при глубоком или широком обходе и усложняет формулировку запросов.

Графовые базы данных хранят связи как объекты первого класса, что позволяет выполнять переходы по графу с постоянной стоимостью на шаг независимо от общего объёма данных. Neo4j — одна из зрелых и широко используемых графовых СУБД — предоставляет декларативный язык Cypher для описания паттернов в графе; запросы на Cypher для поиска связанных наблюдений, цепочек взаимодействий или наиболее влиятельных пользователей обычно короче и читабельнее эквивалентов на SQL с рекурсивными CTE.

Приложение «Animal» содержит тысячи постов, пользователей, тегов и мест, а также множество перекрёстных связей (комментарии, лайки, повторное использование тегов и мест), что делает его практическим примером для демонстрации преимуществ графовых СУБД при работе с сетевыми и иерархическими данными.

Цель работы — разработка веб-приложения для хранения, визуализации и анализа данных наблюдений с использованием Neo4j. Приложение обеспечивает полный цикл работы с данными: импорт/экспорт в

машиночитаемых форматах (JSON), ручное добавление и редактирование постов и связанных сущностей, поиск и фильтрацию по типу, тегам, автору и диапазону дат, интерактивную визуализацию связей в виде графа и формирование статистики использования.

Выбранный стек технологий: Neo4j Community Edition в качестве СУБД, Python с FastAPI для REST API, React для фронтенда. Все компоненты контейнеризированы через Docker и оркестрированы с помощью Docker Compose для упрощённого развёртывания и воспроизводимости среды.

В пояснительной записке описаны постановка задачи, макет и сценарии использования, модель данных (графовая vs реляционная) с их сравнительным анализом, архитектура и применённые технологии, а также выводы по результатам реализации.

1. Постановка задачи

Необходимо разработать веб-приложение для хранения, визуализации и анализа данных наблюдений с использованием графовой СУБД Neo4j.

1.1. Предлагаемое решение

Предлагаемое решение — трёхзвенное веб-приложение, состоящее из:

Neo4j 5.18 Community Edition в качестве графовой БД для хранения сущностей (посты, пользователи, места, теги, комментарии, лайки) и связей между ними в виде узлов и ребер.

Серверная часть (backend) на Python 3.12 с FastAPI, предоставляющая REST API для всех операций над данными (CRUD для постов, пользователей, тегов, мест; операции импорта/экспорта; поиск и фильтрация).

Клиентская часть (frontend) — одностраничное приложение (SPA) на React 19 с Vite 8, реализующее интерфейсы и сценарии использования: страницы входа/регистрации, лента постов с фильтрами, пошаговое добавление наблюдения, профиль пользователя, детальный просмотр поста и интерактивная визуализация связей.

Все компоненты контейнеризированы с Docker и оркестрируются через Docker Compose — одноразовый запуск для развертывания среды.

1.2. Качественные требования к решению

Удобство использования: интуитивный интерфейс с навигацией (Лента / Профиль), интерактивная визуализация связей (масштабирование, перетаскивание), модальные окна для быстрого просмотра и редактирования информации без перехода между страницами.

Контейнеризация: запуск всех компонентов (Neo4j, backend, frontend) одной командой через Docker Compose без ручной установки зависимостей.

Импорт/экспорт данных: поддержка форматов JSON для совместимости с внешними системами и возможностью массового импорта/экспорта ленты (для админов).

Целостность данных: валидация на клиенте и сервере (форматы полей, обязательные атрибуты), каскадное удаление связанных сущностей при удалении поста/пользователя, предзаполнение базы тестовыми данными для демонстрации.

Функциональные требования (кратко): CRUD для постов и комментариев, поиск и фильтрация ленты по типу, тегам, автору и диапазону дат, управление профилем пользователя, лайки/комментарии, статистика и экспорт/импорт.

Производительность и масштабируемость: оптимизация запросов Cypher для типичных сценариев (фильтрация ленты, выборка постов пользователя, подсчёт статистики), использование индексов в Neo4j для ускорения поиска.

Безопасность и отказоустойчивость: базовая аутентификация/авторизация для операций редактирования и админских функций, корректная обработка ошибок и возврат валидных HTTP-статусов.

2. Сценарии использования

Экраны

Страница логина

Поле Email

Поле Пароль

Кнопка «Войти»

Ссылка «Нет аккаунта? Зарегистрироваться»

1а. Страница логина — просмотр пароля

Поле Пароль в режиме отображения текста

Иконка глаза справа в поле — активна

Страница регистрации

Поле Email

Поле Пароль

Кнопка «Зарегистрироваться»

Ссылка «Уже есть аккаунт? Войти»

Лента постов (главный экран после входа)

Хедер с навигацией: Лента / Профиль

Кнопки массового Экспорт / Импорт ленты в формате *.json для админов

Блок профиля пользователя с кнопкой «Выйти»

Кнопка «Добавить» — переход к экрану 3b.1

Кнопка «Фильтры» — открывает экран 3*

Список карточек наблюдений — каждая карточка ведёт к экрану 5

3а. Фильтры (панель фильтрации и сортировки ленты)

Фильтр по типу животного

Сортировка по: тегу, автору, дате

Диапазон дат: с — по

Название места наблюдения

3b.1. Создание наблюдения — шаг 1

Выбор типа животного

Поле Заголовок

Поле Описание наблюдения

Кнопка «Далее» — переход к экрану 3b.2

3b.2. Создание наблюдения — шаг 2

Загрузка фото

Поле Место наблюдения

Поле Теги с кнопкой навешивания тегов

Кнопка «Опубликовать» — публикация и возврат к экрану 3

Кнопка «Назад» — возврат к экрану 3b.1

Профиль пользователя

Информация о пользователе: имя, «о себе», местоположение

Кнопка «Изменить профиль» — редактирование полей «о себе» и «местоположение»

Список наблюдений, опубликованных пользователем — каждое ведёт к экрану 5

Страница поста (детальный просмотр наблюдения)

Тип животного, заголовок, описание, фото, место, теги, автор

Кнопка «Назад»

Кнопка «Лайк»

Секция комментариев с полем ввода и кнопкой отправки

Для постов текущего пользователя: кнопка «Удалить пост» и кнопки «Удалить комментарий» у каждого комментария

Сценарии использования

UseCase-01 — Вход в аккаунт

Экраны: 1 → 1a → 3

Открывает приложение — отображается экран 1.

Вводит email — поле заполняется.

Вводит пароль — символы скрыты.

Нажимает иконку глаза — пароль отображается текстом (экран 1a).

Нажимает иконку снова — пароль скрыт*

Нажимает «Войти» — валидация и переход к экрану 3.

UseCase-02 — Переход к регистрации

Экраны: 1 → 2

На экране 1 нажимает «Нет аккаунта? Зарегистрироваться» — переход к экрану 2.

UseCase-03 — Регистрация нового пользователя

Экраны: 2 → 3

Заполняет email и пароль.

Нажимает «Зарегистрироваться» — аккаунт создан, переход к экрану 3*

UseCase-04 — Возврат к логину со страницы регистрации

Экраны: 2 → 1

Нажимает «Уже есть аккаунт? Войти» — возвращение к экрану 1*

UseCase-05 — Фильтрация ленты

Экраны: 3 → 3a → 3

На экране 3 открывает «Фильтры».

Устанавливает тип животного, сортировку, диапазон дат, место.

Применяет фильтры — возврат к экрану 3 с отфильтрованной лентой*

UseCase-06 — Создание наблюдения

Экраны: 3 → 3b.1 → 3b.2 → 3

Нажимает «Добавить» — переход к 3b.1.

Заполняет тип животного, заголовок, описание — нажимает «Далее».

На 3b.2 загружает фото, указывает место и теги.

Нажимает «Опубликовать» — наблюдение публикуется, возврат к 3*

UseCase-07 — Возврат на шаг 1 при создании

Экраны: 3b.2 → 3b.1

На 3b.2 нажимает «Назад» — возврат к 3b.1.

UseCase-08 — Переход в профиль

Экраны: 3 → 4

На экране 3 нажимает «Профиль» в хедере — переход к экрану 4*

UseCase-09 — Редактирование профиля

Экраны: 4 → редактирование → 4

Нажимает «Изменить профиль» — поля становятся редактируемыми.

Вносит изменения и сохраняет — профиль обновлён*

UseCase-10 — Просмотр поста из ленты

Экраны: 3 → 5

На экране 3 нажимает карточку — переход к экрану 5*

UseCase-11 — Просмотр поста из профиля

Экраны: 4 → 5

В профиле нажимает наблюдение — переход к экрану 5*

UseCase-12 — Лайк наблюдения

Экран: 5

Нажимает «Лайк» — счётчик обновляется*

UseCase-13 — Оставить комментарий

Экран: 5

Вводит текст в поле комментария и нажимает «Отправить» — комментарий добавлен*

UseCase-14 — Удаление своего поста

Экраны: 5 → 3

На своём посте нажимает «Удалить пост» — пост удалён, возврат к 3*

UseCase-15 — Удаление комментария на своём посте

Экран: 5

Нажимает «Удалить» у нужного комментария — комментарий удалён*

UseCase-16 — Возврат из поста

Экраны: 5 → 3 или 4

Нажимает «Назад» — возврат к экрану, откуда открыт пост*

UseCase-17 — Выход из аккаунта

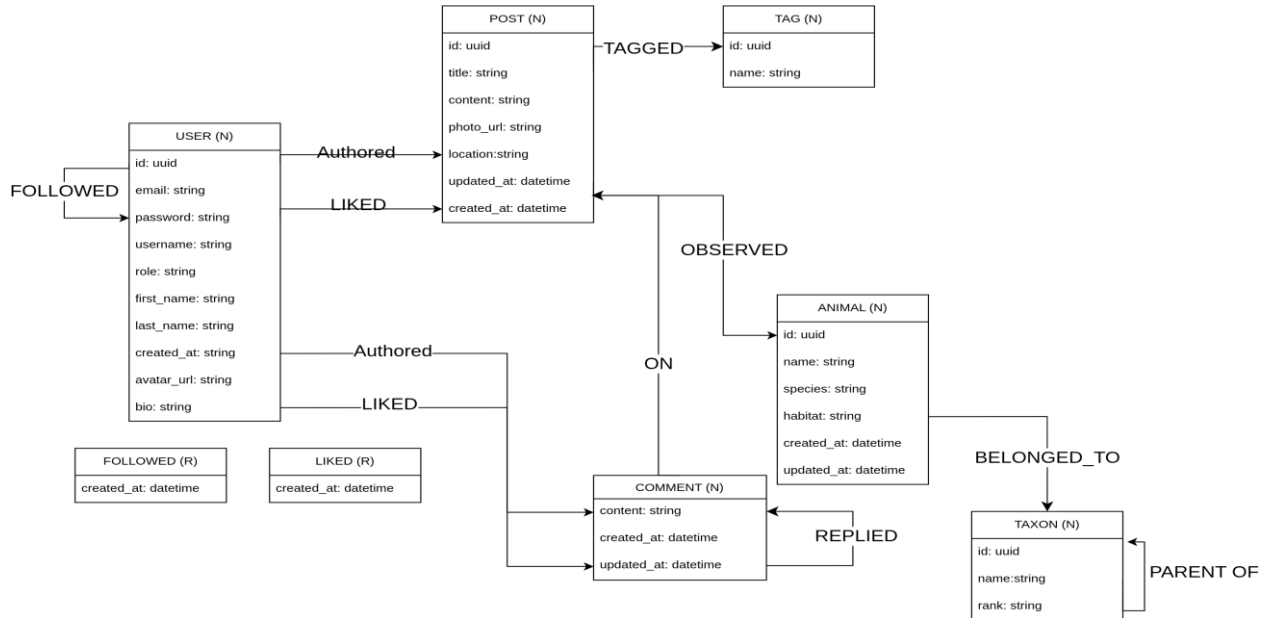
Экраны: 3 → 1

В блоке профиля нажимает «Выйти» — сессия завершена, переход к экрану 1*

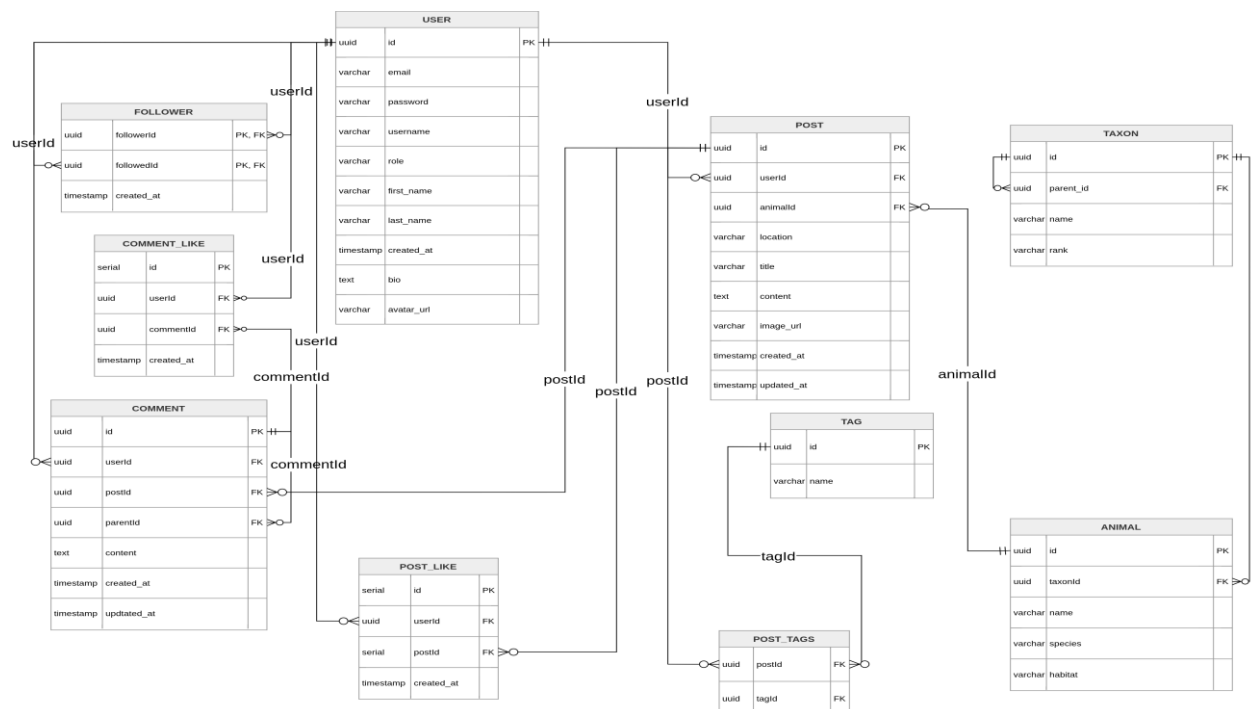
UseCase-18 — Просмотр статистики

3. Модель данных

Нереляционная модель (Neo4j)



Реляционная модель (PostgreSQL)



Сравнение моделей

Use Case / Query Description	SQL Query
1. Retrieve all records from the 'employees' table.	<code>SELECT * FROM employees;</code>
2. Retrieve the first 10 records from the 'employees' table.	<code>SELECT * FROM employees LIMIT 10;</code>
3. Retrieve records from the 'employees' table where the salary is greater than 50,000.	<code>SELECT * FROM employees WHERE salary > 50000;</code>
4. Retrieve records from the 'employees' table where the department is 'Sales'.	<code>SELECT * FROM employees WHERE department = 'Sales';</code>
5. Retrieve records from the 'employees' table where the salary is between 40,000 and 60,000.	<code>SELECT * FROM employees WHERE salary BETWEEN 40000 AND 60000;</code>
6. Retrieve records from the 'employees' table where the name starts with 'A'.	<code>SELECT * FROM employees WHERE name LIKE 'A%';</code>
7. Retrieve records from the 'employees' table where the name contains 'Smith'.	<code>SELECT * FROM employees WHERE name LIKE '%Smith%';</code>
8. Retrieve records from the 'employees' table where the name ends with 'D'.	<code>SELECT * FROM employees WHERE name LIKE '%D';</code>
9. Retrieve records from the 'employees' table where the name is 'John' and the salary is greater than 50,000.	<code>SELECT * FROM employees WHERE name = 'John' AND salary > 50000;</code>
10. Retrieve records from the 'employees' table where the name is 'John' or the salary is greater than 50,000.	<code>SELECT * FROM employees WHERE name = 'John' OR salary > 50000;</code>
11. Retrieve records from the 'employees' table where the name is 'John' and the salary is greater than 50,000, ordered by salary in descending order.	<code>SELECT * FROM employees WHERE name = 'John' AND salary > 50000 ORDER BY salary DESC;</code>
12. Retrieve records from the 'employees' table where the name is 'John' and the salary is greater than 50,000, ordered by salary in descending order, limited to 5 records.	<code>SELECT * FROM employees WHERE name = 'John' AND salary > 50000 ORDER BY salary DESC LIMIT 5;</code>
13. Retrieve the average salary from the 'employees' table.	<code>SELECT AVG(salary) FROM employees;</code>
14. Retrieve the maximum salary from the 'employees' table.	<code>SELECT MAX(salary) FROM employees;</code>
15. Retrieve the minimum salary from the 'employees' table.	<code>SELECT MIN(salary) FROM employees;</code>
16. Retrieve the count of records from the 'employees' table.	<code>SELECT COUNT(*) FROM employees;</code>
17. Retrieve the count of records from the 'employees' table where the salary is greater than 50,000.	<code>SELECT COUNT(*) FROM employees WHERE salary > 50000;</code>
18. Retrieve the count of records from the 'employees' table where the department is 'Sales'.	<code>SELECT COUNT(*) FROM employees WHERE department = 'Sales';</code>
19. Retrieve the count of records from the 'employees' table where the salary is between 40,000 and 60,000.	<code>SELECT COUNT(*) FROM employees WHERE salary BETWEEN 40000 AND 60000;</code>
20. Retrieve the count of records from the 'employees' table where the name starts with 'A'.	<code>SELECT COUNT(*) FROM employees WHERE name LIKE 'A%';</code>
21. Retrieve the count of records from the 'employees' table where the name contains 'Smith'.	<code>SELECT COUNT(*) FROM employees WHERE name LIKE '%Smith%';</code>
22. Retrieve the count of records from the 'employees' table where the name ends with 'D'.	<code>SELECT COUNT(*) FROM employees WHERE name LIKE '%D';</code>
23. Retrieve the count of records from the 'employees' table where the name is 'John' and the salary is greater than 50,000.	<code>SELECT COUNT(*) FROM employees WHERE name = 'John' AND salary > 50000;</code>
24. Retrieve the count of records from the 'employees' table where the name is 'John' or the salary is greater than 50,000.	<code>SELECT COUNT(*) FROM employees WHERE name = 'John' OR salary > 50000;</code>
25. Retrieve the count of records from the 'employees' table where the name is 'John' and the salary is greater than 50,000, ordered by salary in descending order.	<code>SELECT COUNT(*) FROM employees WHERE name = 'John' AND salary > 50000 ORDER BY salary DESC;</code>
26. Retrieve the count of records from the 'employees' table where the name is 'John' and the salary is greater than 50,000, ordered by salary in descending order, limited to 5 records.	<code>SELECT COUNT(*) FROM employees WHERE name = 'John' AND salary > 50000 ORDER BY salary DESC LIMIT 5;</code>
27. Retrieve the average salary from the 'employees' table, grouped by department.	<code>SELECT department, AVG(salary) FROM employees GROUP BY department;</code>
28. Retrieve the maximum salary from the 'employees' table, grouped by department.	<code>SELECT department, MAX(salary) FROM employees GROUP BY department;</code>
29. Retrieve the minimum salary from the 'employees' table, grouped by department.	<code>SELECT department, MIN(salary) FROM employees GROUP BY department;</code>
30. Retrieve the count of records from the 'employees' table, grouped by department.	<code>SELECT department, COUNT(*) FROM employees GROUP BY department;</code>

Postgres (ms)

Neo4j (ms)

UC-01: Login (Find User by Email)	1.907	8.160
UC-05: Filter Feed	1.605	15.514
UC-06: Social Recommendations	0.975	6.576
UC-07: Deep Taxon Tree Search (Recursive)	2.144	5.921
UC-10: Post Detail	1.967	7.188
UC-19: Social Recommendations (FOF)	1.342	6.230
UC-20: Taxonomy Search (Recursive Tree)	1.988	6.137

4. Разработанное приложение

4.1. Краткое описание

Приложение реализовано в виде трехконтейнерной архитектуры, управляемой через Docker Compose:

1) db -- контейнер с графовой СУБД Neo4j 5.18 Community Edition. Хранит все данные о персонах и связях между ними. Данные персистируются через Docker volume.

2) backend -- контейнер с серверной частью на Python 3.12 и фреймворке FastAPI. Предоставляет REST API для всех операций: CRUD-операции над персонами, поиск, статистика, импорт/экспорт. Взаимодействует с Neo4j через официальный Python-драйвер.

3) frontend -- контейнер с клиентской частью на React 19 и Vite 8. Реализует одностраничное приложение (SPA) с маршрутизацией через react-router-dom. Граф родословного дерева отрисовывается с помощью Canvas API.

4.2. Используемые технологии

Компонент	Технологии
Backend	Python 3.12, FastAPI 0.111, Uvicorn 0.29, neo4j 5.26 (драйвер), Pydantic 2.7, pydantic-settings 2.2
Frontend	React 19, Vite 8, react-router-dom 7, Canvas API (визуализация графа)
База данных	Neo4j 5.18 Community Edition
Инфраструктура	Docker, Docker Compose
CI/CD	GitHub Actions (7 workflow-проверок)

4.3. Снимки экрана приложения

Описание экранов приложения со снимками представлено в Приложении Б (Инструкция для пользователя).

5. Выводы

Реализовано веб-приложение для хранения, визуализации и анализа родословной династии Романовых; поддержаны 9 из 10 сценариев (UC-05 — кастомная статистика не реализована); база содержит 182 персоны и 766 связей; приложение обеспечивает поиск, редактирование, импорт/экспорт и интерактивную визуализацию графа; в планах — добавить настраиваемую статистику, улучшить работу с мультимедиа и оптимизировать визуализацию для больших графов.

Приложение А. Документация по сборке и развертыванию

Предварительные требования

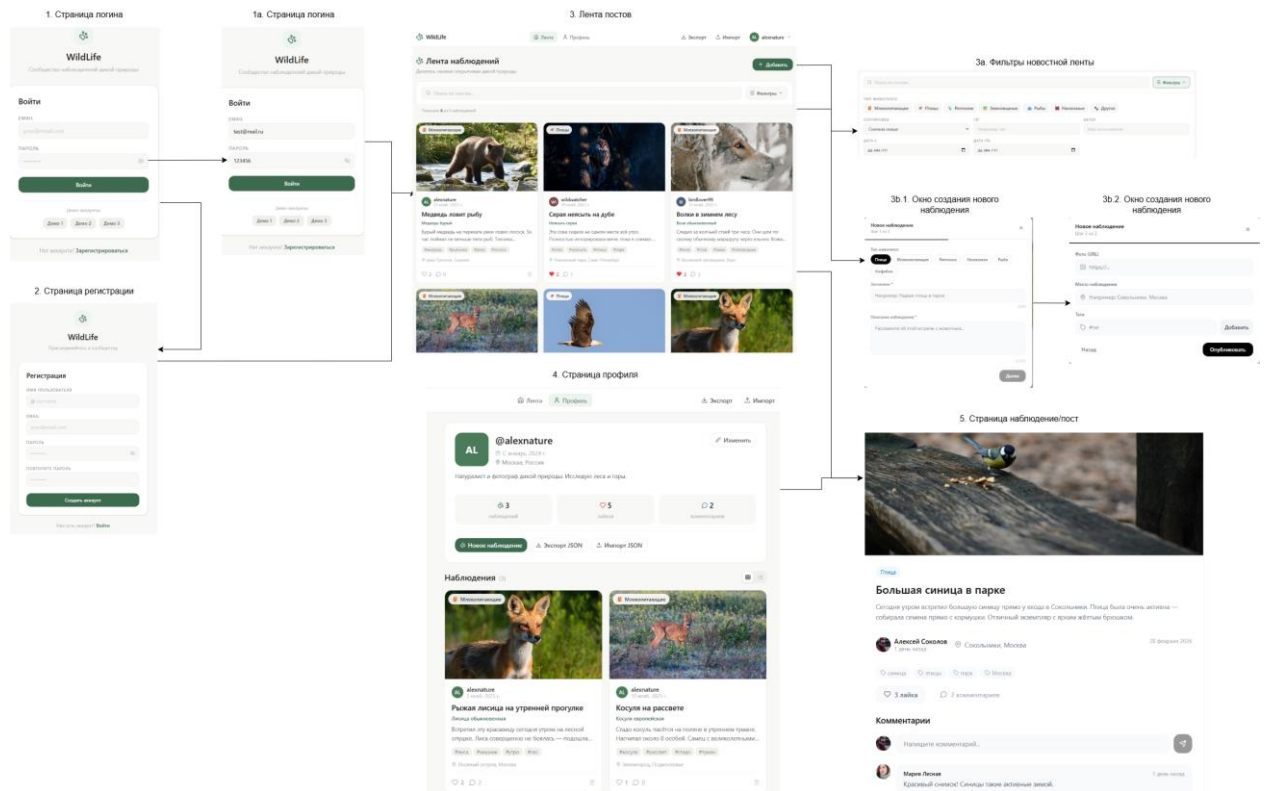
Для развертывания приложения необходимо наличие установленных Docker и Docker Compose.

Порядок развертывания

1. Собрать и запустить контейнеры: `docker compose build --no-cache && docker compose up`
2. Открыть Swagger API:
`http://localhost:8000/api/v1/docs`
3. Открыть фронтенд:
`http://localhost:5173`
4. Регистрация/вход:
Через фронтенд: зарегистрируйтесь (например, email: `string@mail.ru`, пароль: 123).
Или через Swagger: `POST /auth/register`, затем используйте куки/токен для авторизации.
5. Тестовые аккаунты:
`vlad2005 / 123`
`kirill2005 / 123`
`alex2005 / 123`
6. Примечания:
Если контейнеры не поднимаются — проверьте, что локальные папки не смонтированы в контейнеры (запрещено).

Приложение Б. Инструкция для пользователя

<https://github.com/moevm/nsql1h26-anim/wiki>



Литература

1. Neo4j Documentation [Электронный ресурс]. -- Режим доступа:
<https://neo4j.com/docs/>
2. FastAPI Documentation [Электронный ресурс]. -- Режим доступа:
<https://fastapi.tiangolo.com/>
3. React Documentation [Электронный ресурс]. -- Режим доступа:
<https://react.dev/>
4. Репозиторий проекта [Электронный ресурс]. -- Режим доступа:
<https://github.com/moevm/nsql1h26-tree>
5. Docker Documentation [Электронный ресурс]. -- Режим доступа:
<https://docs.docker.com/>
6. Vite Documentation [Электронный ресурс]. -- Режим доступа:
<https://vite.dev/>
7. Cypher Query Language [Электронный ресурс]. -- Режим доступа:
<https://neo4j.com/docs/cypher-manual/current/>